

Module 3: Design, Cohesion & Coupling

Modularity Scales, SOLID Principles, GoF Design Patterns (Singleton, Factory, Observer)

☰ **Module III: Software Design Engineering & UML — Complete 13-Topic Tracker**

- Topic 27:** Design Engineering (Architecture vs. Detailed Component Design)

Topic 29: Software Design Quality Attributes (Modularity & Maintainability)

Topic 31: Design Models (Data, Architectural, Interface & Component)

Topic 33: Cohesion (7 Levels: Coincidental to Functional Cohesion)

Topic 35: UML Use Case Diagrams (Actors, Systems & Include/Extend)

Topic 37: UML Activity Diagrams (Workflows, Swimlanes & Fork/Join)

Topic 39: UML Collaboration / Communication Diagrams
- Topic 28:** The Software Design Process (Iterative Model Transformations)

Topic 30: Fundamental Design Concepts (Abstraction & Information Hiding)

Topic 32: Object-Oriented Design Principles (SOLID Design Principles)

Topic 34: Coupling (5 Levels: Content to Data Coupling)

Topic 36: UML Class Diagrams (Attributes, Methods, Visibility & Relations)

Topic 38: UML Sequence Diagrams (Lifelines, Messages & Time Sequence)

Topic 27 – 30: Fundamental Software Design Concepts

Design Concept	Engineering Principle & Definition	Key Benefit
1. Abstraction	Suppressing low-level implementation details to focus on high-level essential characteristics.	Reduces cognitive complexity for system architects.
2. Modularity	Decomposing software into discrete, independently testable, naming-addressable modules.	Enables parallel team development and isolated debugging.
3. Information Hiding	Modules are designed so that internal algorithms and data structures are inaccessible from outside (Parnas, 1972).	Prevents cascading bug propagation when internal code changes.
4. Functional Independence	Achieved by designing modules with High Cohesion and Low Coupling .	Maximizes code reusability and minimizes side-effect bugs.

Topic 33 & 34: Cohesion vs. Coupling (The Cornerstone of Modular Design)

Cohesion Level (Internal Focus)	Operating Semantics	Quality Rating
1. Functional Cohesion	Every instruction in the module contributes to executing exactly ONE single mathematical/business function (e.g., <code>calculate_tax()</code>).	★★★★★ BEST
2. Sequential Cohesion	Output data from one operation serves as direct input to the next operation in the module.	★★★★ Very Good
3. Communicational Cohesion	Functions operate on the exact same input data structure (e.g., <code>print_student_record()</code> and <code>save_student_record()</code>).	★★★ Good
4. Procedural / Temporal	Functions executed in a sequence or executed at the same time (e.g., <code>startup_initialization()</code>).	★★ Moderate
5. Coincidental Cohesion	Arbitrary unrelated tasks grouped together in a <code>utils.py</code> file with no logical relationship.	★ WORST

Coupling Level (Inter-Dependency)	Operating Semantics	Quality Rating
1. Data Coupling	Modules communicate strictly by passing necessary simple primitive data parameters (x, y).	★★★★★ BEST
2. Stamp Coupling	Modules pass entire complex data structures when only a single field is actually needed.	★★★★ Good
3. Control Coupling	One module passes a control flag (<code>bool is_admin</code>) that dictates the internal execution path of another.	★★ Moderate
4. Common Coupling	Multiple modules read and write to shared global variables.	★ Poor
5. Content Coupling	One module directly accesses, modifies, or branches into the internal memory/code of another module.	✖ WORST

🧠 Golden Rule of Software Design

HIGH COHESION (Functional) + LOW COUPLING (Data) = Publication-Grade Software Architecture!

Topic 35 – 39: Unified Modeling Language (UML) Diagrams

UML Diagram	Structural / Behavioral	Key Modeling Elements & Purpose
1. Use Case Diagram	Behavioral	Actors (stick figures), Use Cases (ovals), System boundary, `<>` (mandatory) and `<>` (optional).
2. Class Diagram	Structural	Classes with 3 compartments: Name, Attributes (`- private`, `+ public`, `# protected`), Operations. Inheritance ($\triangle$), Association, Composition ($\blacklozenge$).
3. Activity Diagram	Behavioral	Initial/Final nodes, Action states, Decision diamonds, Swimlanes, Fork/Join synchronization bars.
4. Sequence Diagram	Behavioral (Dynamic)	Lifelines (vertical dashed lines), Activation boxes, Synchronous solid-arrow calls, Asynchronous open-arrow calls, Return dashed arrows.
5. Collaboration Diagram	Behavioral (Dynamic)	Emphasizes structural organization of objects with numbered message arrows (1.0, 1.1) rather than a vertical time axis.

M3 Active Recall & Exam Questions

Q1. Differentiate between `<>` and `<>` relationships in UML Use Case Diagrams with examples. (8 Marks)

- `<>` Relationship: Represents mandatory common functionality unconditionally shared across multiple use cases. The base use case *cannot execute* without invoking the included use case.

Example: `PlaceOrder` $\rightarrow$ {>} `AuthenticateUser`.

- `<>` Relationship: Represents optional, conditional behavior that executes only when a specific extension point condition is met at runtime.

Example: `CalculateTotal` $\leftarrow$ {>} `ApplyDiscountCoupon` (executed only if the customer possesses a valid coupon code).